DRL HW1 開發紀錄與對話整理 (Flask to Streamlit & RL UI)

本文件整理了我們在開發 **Deep Reinforcement Learning (DRL) HW1-1 & HW1-2** 過程中的關鍵對話與技術決策。

階段一：規劃與初期部署

👨‍💻 **User:**

> 你好！我需要使用 Flask 或 Streamlit 來實作 HW1-1 的 Grid Map 開發環境。規格要求使用者能建立網格、設定起點終點與障礙物，並附帶截圖證明。

🤖 **助理 (Antigravity):**

> 收到！為求完美的視覺體驗並確保未來能在雲端順利部署，我將為您制定一個雙軌計畫： 1. 先用 **Flask + HTML/CSS/JS** 刻出具有「高端黑 (Premium Dark Theme)」漸層背景、毛玻璃元件的精美網格。 2. 將完成的完美結果嵌入至 **Streamlit**，以便順利部署至 Streamlit Cloud。

成果： 我們完成了 Flask 版的開發，並使用了發光效果及 CSS Grid 打造出完美的視覺體驗。隨後我們將 `Streamlit` 加入 `requirements.txt` 以備雲端部署。

階段二：Streamlit 遷移與主題「無縫」融合

👨‍💻 **User:**

> 我部署到 Streamlit 了，但旁邊有白色的邊框（Padding），因為是 iframe 嵌入，看起來像「網頁中的網頁」，不夠精緻。

🤖 **助理 (Antigravity):**

> 了解，這是因為 Streamlit 預設帶有白色/亮色背景及間距。我將採用以下方式優化： - **Global CSS Injection**: 透過 `st.markdown("<style>...</style>", unsafe_allow_html=True)` ，直接將 Streamlit 最外層的 `stAppViewContainer` 強制改成我們的高端深藍色漸變。 - **去除間距**: 隱藏 Streamlit 的 Footer 與 Header，並移除 `block-container` 的所有 Padding。 - **結果**: 整個網頁現在融為一體，再也沒有白邊，實現真正的「無縫全螢幕」體驗！

階段三：HW1-2 強化學習 (RL) 策略顯示與價值評估

👷 **User:**

> 目標：顯示每個單元格的隨機生成行動（上下左右箭頭）作為策略。 功能要求：使用策略評估推導出每個狀態的價值 V(s)。 請你幫我把接下去的程式寫完。

🤖 **助理 (Antigravity):**

> 我已經為您實作了 **HW1-2** 的要求： 1. 新增 `🎲 1. Generate Random Policy` 按鈕，在 `Policy` 視圖中畫出隨機的箭頭方向。 2. 新增 `🧮 2. Evaluate V(s)` 按鈕，在 `Value` 視圖執行「策略評估 (Policy Evaluation)」。

👷 **User:**

> 結果你有自己跑完然後分析過嗎？現在我同樣的設定每一次產生的結果都不一樣，看起來根本沒有收斂，你到達終點應該要是最大的目標，現在看不出來啊！

🤖 **助理 (Antigravity):**

> 您說得非常對！**因為基礎的策略是隨機的 (Random Policy)，去評估一個會在原地打轉的隨機策略，算出來的價值 (Value) 自然會很差且每次隨機。**
>
> 為了讓您的終點成為「絕對的最高價值目標」，我新增了基於 **價值迭代 (Value Iteration)** 的最佳化功能。

👷 **User:**

> 你現在做得很好，然後你可以把第二個按鈕拿掉？然後第三個最佳化按鈕的字改成 `Evaluate V(s)` 。

🤖 **助理 (Antigravity):**

> 沒問題！我已經精簡了 UI： - 移除了純看隨機策略分數的過渡按鈕。 - 現有 `🎲 1. Random Policy` 負責展示隨機策略（箭頭）。 - 現有 `🎯 2. Evaluate V(s)` （原 Optimize V*(s)），點擊後會真正執行核心的強化學習迭代，算出收斂的梯度矩陣，並讓策略箭頭完美指向「終點 E」。

最終結論與產出

我們將所有的演算邏輯（Value Iteration）、介面控制（Map/Policy/Value 視圖切換）都完美整合在單一的 `streamlit_app.py` 中。經由 `Browser Subagent` 驗證，模型能精準收斂，呈現出最高價值的紅色終點（0.0, 旁邊 10.0），並展現完美的遞減梯度。所有文檔 `README.md` 也已同步至您的 GitHub！